

VETORES E MATRIZES, CADEIAS DE CARACTERES, ESTRUTURAS E TABELAS DE ESTRUTURAS

5. Vetores e matrizes

5.1. *Nível básico*

1. Construa uma função que substitua, num vetor de inteiros dado, todos os números ímpares pelo seu dobro. Teste esta função criando um programa que leia um vetor de inteiros da consola e escreva no ecrã o vetor transformado pela função que escreveu.
2. Construa uma função que, dados dois vetores de double a e b, calcule o vetor-soma e o devolva num vetor c. Teste esta função criando um programa que leia dois vetores da consola e escreva no ecrã o seu vetor-soma.
3. Escreva um programa que crie um vetor composto por 10 inteiros lidos da consola. Em seguida, crie um novo vetor com idêntico tamanho e copie os inteiros do 1º vetor, mas por ordem inversa. Finalmente, apresente os números do novo vetor no ecrã.
4. Escreva um programa que leia um conjunto de dez inteiros, que os armazene numa estrutura de dados adequada, e determine o maior valor dos últimos seis elementos. Deverá modularizar o seu programa construindo e utilizando as seguintes funções:
 - Uma função que leia n inteiros e os carregue num vetor dado;
 - Uma função que calcule o valor máximo de um vetor de inteiros dado.
5. Escreva um programa que crie um vetor com os primeiros N números primos. O número N deve ser pedido ao utilizador. No final, o conteúdo do vetor deve ser mostrado na consola.
6. Crie uma função que, dada uma tabela bidimensional de inteiros, a visualize no ecrã.
7. Faça uma função que receba uma tabela bidimensional e determine a transposição da matriz recebida. Mostre a matriz resultante no ecrã recorrendo à função construída no exercício anterior.
8. Construa um programa que crie a seguinte tabela com distâncias entre 5 cidades:

Cidades	1	2	3	4	5
1	0	20	30	40	50

Princípios de Programação Procedimental

2	20	0	20	30	40
3	30	20	0	20	30
4	40	30	20	0	20
5	50	40	30	20	0

O programa deverá em seguida entrar num ciclo onde, em cada iteração, pergunta duas cidades ao utilizador e devolve a distância entre elas. O ciclo deverá terminar quando o utilizador assim o desejar.

9. Faça um programa que construa uma matriz de $N \times N$ (N é fixado pelo programador, no início do programa) valores inteiros. Desenvolva, ainda, as seguintes funções:

- Uma função que peça ao utilizador os valores necessários para preencher a matriz;
- Uma função que construa uma matriz resultante da inicial multiplicada por um valor pedido ao utilizador;
- Uma função que receba uma matriz, o número de linhas e o número de colunas, e escreva a matriz no ecrã.

Desenvolva o código necessário para testar todas as funcionalidades descritas.

5.2. Nível médio

1. Implemente um programa em C que pergunte ao utilizador o número n de elementos com que deseja trabalhar, e depois permita ao utilizador efetuar as seguintes operações:

- Introduzir uma sequência de n números inteiros.
- Escrever no ecrã a sequência introduzida.
- Procurar o maior número da sequência.
- Procurar o menor número da sequência.
- Calcular o somatório da sequência.
- Calcular a média da sequência.
- Determinar a subsequência abaixo da média.
- Determinar a subsequência acima da média.
- Terminar o programa.

Princípios de Programação Procedimental

O programa deverá fazer uso extenso e apropriado de funções de modo a não repetir código.

2. Faça um programa que leia uma tabela unidimensional com N componentes – N a definir pelo utilizador - e para cada elemento, realize a troca do seguinte com o anterior se estes forem, respetivamente, menor e maior que o elemento. Escreva uma função para fazer as trocas na tabela. Mande escrever os resultados no ecrã.

Exemplos:

[.. 8, 7, 3, ...] * TROCA * [.. 3, 7, 8, ...]

[.. 3, 4, 2, ...] * Não TROCA * [.. 3, 4, 2, ...]

3. Escreva um programa que construa dois vetores, v1 e v2, com inteiros lidos da consola (até um máximo de 10) e em seguida ordene cada um deles por valores crescentes. Depois, o programa deverá construir um novo vetor, total, ordenado por valores crescentes, composto por todos os elementos dos dois outros vetores. A construção do novo vetor deverá seguir um algoritmo que tire partido do facto de os vetores v1 e v2 já estarem ordenados. No final, o conteúdo do vetor total deve ser mostrado na consola.
4. Escreva um programa que peça ao utilizador N valores e imprima esses mesmos valores sem duplicados, isto é, se o utilizador inseriu os números 1, 2, 2, então o programa só imprimirá os números 1 e 2, visto que são elementos distintos. Utilize ordenação de vetores, notando que, ao utilizar um vetor ordenado, já não é necessário percorrer SEMPRE todos os seus elementos para determinar se um valor é repetido ou não.
5. Faça uma função que receba duas tabelas de inteiros bidimensionais e determine o produto das duas matrizes recebidas. Mostre a matriz resultante no ecrã recorrendo à função construída no exercício 6.

5.3. Nível avançado

1. Faça um programa que peça ao utilizador as dimensões de uma tabela bidimensional de inteiros, bem como os valores para o seu preenchimento. De seguida, o seu programa deverá ordenar todos os valores da tabela, de acordo com o exemplo a seguir:

Matriz original:

8	2	5	10
12	4	1	9
3	7	11	6

Princípios de Programação Procedimental

Matriz ordenada:

1	4	7	10
2	5	8	11
3	6	9	12

O programa deverá apresentar ambas as matrizes, original e ordenada, no ecrã.

6. Strings ou cadeias de caracteres

6.1. *Nível básico*

1. Escreva uma função que determine o número de caracteres de uma string, sem usar a função `strlen`.
2. Escreva uma função que dadas duas strings copie o conteúdo da primeira para a segunda, sem usar a função `strcpy`.
3. Escreva uma função que dadas duas strings `a` e `b` procure a primeira ocorrência de `b` em `a`.
4. Escreva uma função que dado um número inteiro o converta em formato string.
5. Escreva uma função que dada uma string contendo um número inteiro determine esse número e o devolva em formato `int`.
6. Escreva um programa que leia uma cadeia com até um máximo de 60 caracteres e determine o número de palavras que a constituem. Admita que as palavras são separadas por espaços apenas. Deverá modularizar o seu programa construindo e utilizando uma função que determine o número de palavras que constituem a string.
7. Escreva um programa que leia um conjunto de caracteres da consola (até '\n' ou um máximo de 20) converta as letras minúsculas em maiúsculas e finalmente imprima o resultado no ecrã.
8. Construa uma função que elimine, numa cadeia de caracteres dada, todos os espaços em branco. Teste esta função criando um programa que leia uma linha de caracteres da consola e escreva a mesma linha com os espaços eliminados.

6.2. *Nível médio*

1. Implemente um programa que receba uma string com o máximo de 50 caracteres e que o codifique utilizando a seguinte tabela de codificação:

```
ABCDEFGHIJKLMNOPQRSTUVWXYZ  
DEIABCFGHJKLZYXWVUTSRQPONM
```

Princípios de Programação Procedimental

Ou seja, cada carácter da primeira linha deverá ser transformado no carácter correspondente da segunda. Note que deverá ter em atenção os seguintes aspetos:

- Deverá converter todos os caracteres da string de entrada em maiúsculas.
- Todos os caracteres não previstos na tabela devem ser mantidos iguais na saída.

De seguida implemente um programa que receba uma string encriptada utilizando a cifra mencionada e que faça o processo inverso obtendo assim a string inicial.

2. O algoritmo de compressão de dados pode ser conseguido por meio da deteção e fusão de dígitos repetidos consecutivamente. A título exemplificativo, a string "123555555555542" seria comprimida, resultando a string "123c512f42". Tal como se pode verificar, o dígito "5", aparece repetido 12 vezes na string original. Deste modo, a sucessão de 12 vezes do dígito 5, é comprimida em "c512f", em que o carácter "c" denota o início da compreensão, o carácter "5" indica que o dígito 5 aparece repetido, os caracteres entre "5" e "f" indicam o número de vezes que o carácter é repetido (neste caso "12") e, finalmente, o carácter "f" indica o fim da compressão. Pretende-se que desenvolva um programa que peça ao utilizador os dados a comprimir (numa string que só pode conter algarismos) e escreva no ecrã a sua equivalente comprimida. O seu programa deve incluir um método compressor que receba uma string não comprimida e devolva a sua versão comprimida.

Nota: A compressão implica a existência de pelo menos 4 dígitos: "c", dígito repetido, número de vezes que é repetido e "f". Assim, só faz sentido efetuar compressões em repetições superiores a 4 dígitos.

3. No exercício anterior foi experimentado um algoritmo simples de encriptação de strings. Com este exercício pretende-se implementar um mecanismo diferente. Antes de introduzir a string, o utilizador introduz um valor inteiro positivo. De seguida, introduz a string. A string introduzida poderá ter qualquer tipo de carácter (letras ou algarismos). O valor inicialmente introduzido será utilizado para a deslocação no alfabeto, bem como nos algarismos. Não existe um limite superior para este valor, no entanto, se este for superior ao tamanho da string deverá ser usado o valor de "n%size" (sendo size o tamanho da string).

Exemplo:

Valor introduzido: 2

String inicial: "Experiencia123"

Resultado intermédio: "Gzrgtkgpekc345"

Uma vez criada a string encriptada (intermédia), esta deve sofrer uma rotação correspondente ao valor utilizado para a encriptação – string final.

Resultado Final: "45Gzrgtkgpek3"

Desta forma, para fazer a descriptação basta ter a string final e o valor numérico utilizado para a encriptação. Implemente também esta funcionalidade no seu programa.

4. Escreva um programa que leia uma cadeia de caracteres até um máximo de 60 caracteres e determine o número ocorrências de uma determinada palavra.
5. Escreva um programa que leia uma cadeia de caracteres até um máximo de 60 caracteres e substitua todas as ocorrências de uma determinada palavra por uma terceira palavra fornecida.

6.3. *Nível avançado*

1. Implemente um programa que permita verificar a ocorrência de um conjunto de palavras numa string fornecida pelo utilizador. O seu programa deverá conter um dicionário com o formato:

```
char dictionary[D_SIZE][20] = {"palavra01", "palavra02", "palavra03"};
```

Nota: D_SIZE é uma constante que indica qual o número de palavras no dicionário. A string a analisar deve ser pedida ao utilizador durante a execução do programa. A string recebida deve ser passado por parâmetro a uma função a definir por si, que irá verificar se as palavras contidas no dicionário estão presentes ou não na string. O programa deve exibir no ecrã todas as palavras encontradas sem repetições, mas indicando o número de vezes que estas ocorreram na string.

2. Faça uma função que receba uma tabela bidimensional de caracteres e determine, na matriz recebida, a ocorrência de uma palavra fornecida. A palavra poderá ocorrer na horizontal, vertical ou na diagonal (direção da diagonal principal).

7. Estruturas e matrizes de estruturas

7.1. *Nível básico*

1. Declare o tipo de dados Data que permita armazenar uma data (dia, mês, ano) numa só variável.
 - crie uma função que compare duas datas. Deve devolver 0 se elas forem iguais, -1 se a primeira for anterior à segunda e 1 se a primeira for posterior à segunda;
 - crie uma função que devolva a diferença entre duas datas (em dias, meses e anos);
 - crie uma função que, recebendo uma data, devolva o século correspondente.

7.2. *Nível médio*

1. Dado um vetor de elementos do tipo Data (contém três inteiros: dia, mês, ano)

Princípios de Programação Procedimental

- crie uma função para ordenar o vetor de datas;
 - crie uma função que diga qual a menor data presente no vetor;
 - desenvolva uma função que crie um novo vetor com todas as datas do vetor inicial que correspondem a um ano que é passado como parâmetro.
2. Num campeonato há 20 equipas em competição. Defina um vetor que contenha os dados dessas equipas, concretamente o nome, o número de jogos jogados, os pontos ganhos. Defina um outro vetor com os resultados dos jogos, que em cada posição tenha o nome das equipas que se defrontaram, e os golos marcados por cada equipa.

Desenvolva:

- uma função para percorrer o vetor de resultados de jogos e calcular os pontos ganhos por cada equipa (vitória - dois pontos, empate - 1 ponto, derrota - 0 pontos), atualizando o vetor com os dados das equipas.
- uma função para ordenar o vetor de equipas por ordem de pontos ganhos. Se houver duas equipas com os mesmos pontos, ordenar em primeiro lugar quem tiver marcado mais golos. Se persistir o empate, ordenar por ordem alfabética.

7.3. *Nível avançado*

1. Um professor pretende uma aplicação para guardar dados sobre os seus alunos. Em relação a cada um pretende guardar o nome, número, data da primeira aula frequentada e as notas obtidas em cada um dos 4 testes efetuados à disciplina. Crie as estruturas de dados necessárias para armazenar esta informação e crie funções para o seguinte:
- inserir os dados de um novo aluno;
 - verificar se existe uma ficha relativa a um aluno com determinado número;
 - escrever o nome e número de aluno de todos os alunos que começaram a frequentar a disciplina antes de uma determinada data;
 - determinar qual o aluno com melhor média nos testes realizados;
 - listar todos os alunos aprovados (soma dos testes maior ou igual a 10 e nenhum teste inferior a 2);
 - listar todos os alunos, ordenando-os por ordem decrescente da sua média.
2. Estenda o problema 2 da secção anterior disponibilizando ao utilizador funcionalidades para:
- - Introduzir, consultar e alterar os dados das equipas;

Princípios de Programação Procedimental

- - Introduzir, consultar e alterar os dados dos jogos (inclua também a data do jogo);
- - Gerar e mostrar a classificação das equipas, considerando como critério de desempate em caso de igualdade pontual a maior diferença entre golos marcados e sofridos e, se persistir o empate, ordenar primeiro quem atingiu primeiro o número atual de pontos e, se ainda persistir o empate, seguir a ordem alfabética.